

Landesberufsschule 4 Salzburg

Übungen im IT - Laboratorium

Übung DB-Analyse

für die Übung Nr. 10

Katalog - Nr.:

4IA

Name :

Vanessa Buchacek

Jahrgang :

2025/26

Datum der Übung :

09.10.2025

1 Inhalt

1	Inhalt.....	2
2	Zusammenfassung der Übung <i>DB-Analyse</i>	3
3	Anweisung der Übung	3
4	Einleitung.....	3
5	Inventarliste.....	3
5.1	Hardware	3
5.2	Software	3
6	Übungsdurchführung	4
6.1	Struktur und Daten	4
6.2	ER-Diagramm mit Reverse-Engineering.....	4
6.3	Fremdschlüsselbeziehungen erstellen.....	5
6.4	Select Abfragen	5
6.4.1	5 Personen anzeigen & sortieren	5
6.4.2	Berufsbezeichnung Sales Rep & OfficeCode 1 auswählen	5
6.4.3	Alle Nachnamen, keine Doppelten, absteigend sortiert	6
6.4.4	USA & Frankreich.....	6
6.4.5	! Zwischen 20&100 Dollar.....	6
6.4.6	Buchstabe B ausblenden	7
6.4.7	Verkaufsvertreter zugewiesen	7
6.4.8	Bestelldaten gruppieren	8
6.4.9	Angestellte ohne zugewiesene Kunden.....	8
6.4.10	Bestellungen unter 1500\$ Gesamtsumme.....	9
6.4.11	SubSelect – Kunde mit dem meisten Umsatz	9
6.4.12	Kunden ohne Bestellung	10
6.4.13	UNION Operator – Kunden und Angestellte 1 Tabelle.....	10
6.5	Neue Tabelle suppliers.....	10
6.5.1	Tabelle mit customers Daten befüllen	11
6.6	Maildomains aktualisieren – replace	11
6.7	Kunden ohne SalesRep zufällige zuweisen	11
7	Erweiterung	12
8	Einsatzgebiet	13
9	Erkenntnisse	13
10	Abbildungsverzeichnis.....	14

2 Zusammenfassung der Übung *DB-Analyse*

In dieser Laborübung habe ich Fremdschlüsselbeziehungen definiert, ein ER-Diagramm mittels Reverse-Engineerings erzeugt und verschiedene SQL-Abfragen (SELECT, UPDATE, JOIN, SubSelect, UNION) angewendet.

3 Anweisung der Übung

- SQL-Skript importieren und Datenbank erstellen
- Reverse-Engineering durchführen und ER-Diagramm erstellen
- Primär-/Fremdschlüsselbeziehungen zwischen customers - employees und products - orderdetails
- Verschiedene SELECT-Abfragen ausführen (Sortierung, Filter, Gruppierung, SubSelects und UNION-Abfragen)
- Neue Tabelle suppliers erstellen und mit Daten aus customers befüllen
- Maildomain aktualisieren mit replace
- Zufällige Zuweisung von salesRepEmployeeNumber

4 Einleitung

In dieser Übung habe ich begonnen das SQL-Skript in MySQL Workbench zu importiert, um die Datenbank automatisch zu erstellen. Danach habe ich mittels Reverse-Engineerings ein ER-Diagramm erstellt, um die Zusammenhänge der Tabellen visuell darzustellen. Anschließend habe ich verschiedene Datenbankabfragen und Bearbeitungen nach den Arbeitsanweisungen durchgeführt.

5 Inventarliste

5.1 Hardware

- Notebook

5.2 Software

- mySQL Workbench
- XAMPP



6 Übungsdurchführung

6.1 Struktur und Daten

Ich habe die gegebene DB- Struktur und Daten als sql-Skript ausgeführt und somit die DB mit allen Daten nötig für dich Aufgaben der Analyse „erstellt“.

6.2 ER-Diagramm mit Reverse-Engineering

Das Reverse-Engineering war mit mySQL-Workbench leicht auszuführen, im Tab Database gibt es eine Option fürs reverse-engineering, wobei ich die neue DB ausgewählt habe, nach der erfolgreichen Verbindung war das ER-Diagramm bereits erstellt.



Abbildung 2 - reverse engineering mysql workbench

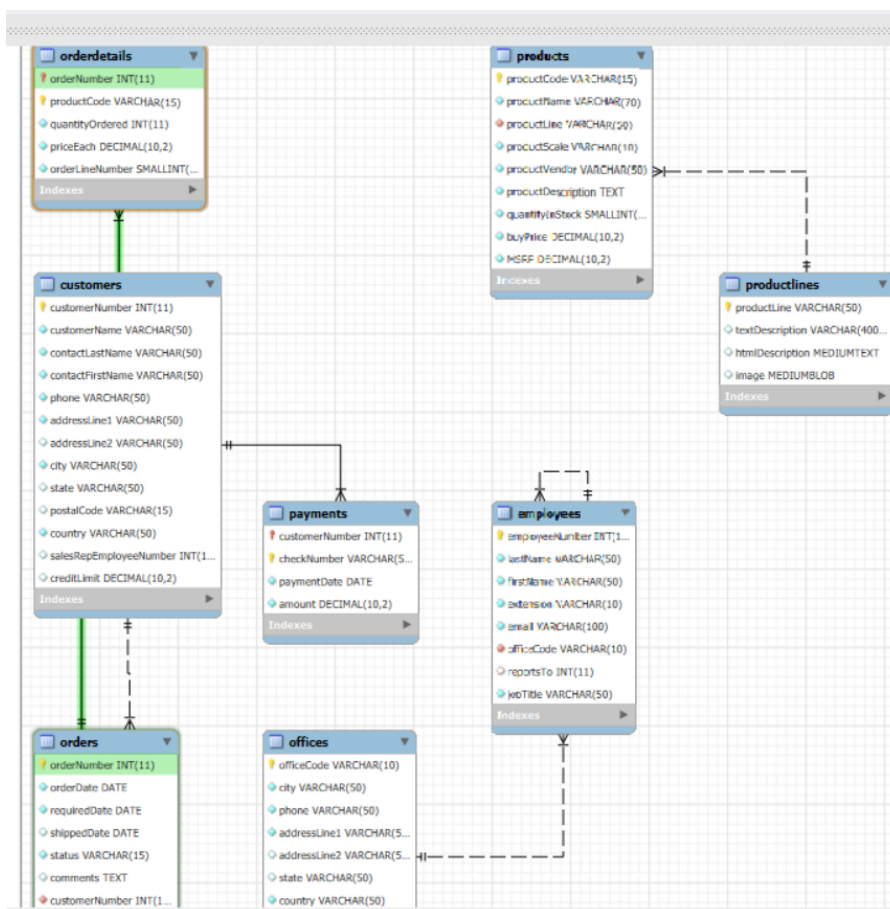


Abbildung 3 - ER Diagramm

6.3 Fremdschlüsselbeziehungen erstellen

Ich habe die benötigten Tabs bearbeitet, um die Fremdschlüssel-Beziehungen zu erstellen. Einen Constraint hinzugefügt und den Foreignkey referenzierend zur EmployeeNumber erstellt.

```
ALTER TABLE customers
ADD CONSTRAINT customers_ibfk_1
FOREIGN KEY (salesRepEmployeeNumber) REFERENCES employees(employeeNumber);

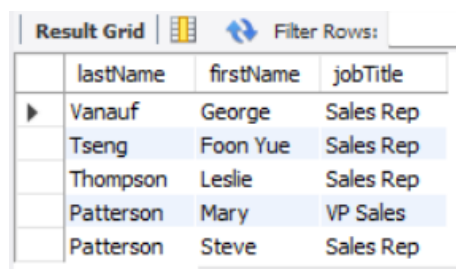
ALTER TABLE orderdetails
ADD CONSTRAINT orderdetails_ibfk_2
FOREIGN KEY (productCode) REFERENCES products(productCode);
```

6.4 Select Abfragen

6.4.1 5 Personen anzeigen & sortieren

Um die ersten 5 Personen aus der employee Tabelle zu selecten und deren Namen und Jobtitel anzuzeigen habe ich diesen Befehl ausgeführt.

```
SELECT lastName, firstName, jobTitle
FROM employees
ORDER BY lastName DESC
LIMIT 5;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains 5 rows of employee data. The first row is highlighted with a mouse cursor.

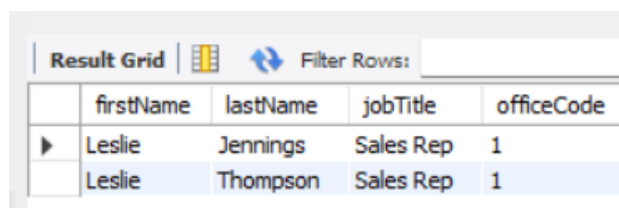
	lastName	firstName	jobTitle
▶	Vanauf	George	Sales Rep
	Tseng	Foon Yue	Sales Rep
	Thompson	Leslie	Sales Rep
	Patterson	Mary	VP Sales
	Patterson	Steve	Sales Rep

Abbildung 4 - 5 Personen Abfrage

6.4.2 Berufsbezeichnung Sales Rep & OfficeCode 1 auswählen

Um Personen aus der DB zu filtern, die die Berufsbezeichnungen SalesRep und officeCode = 1 haben, reicht ein einfacher select:

```
SELECT firstName, lastName, jobTitle, officeCode
FROM employees
WHERE jobTitle = 'Sales Rep' AND officeCode = 1;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. The grid contains 2 rows of employee data. The first row is highlighted with a mouse cursor.

	firstName	lastName	jobTitle	officeCode
▶	Leslie	Jennings	Sales Rep	1
	Leslie	Thompson	Sales Rep	1

Abbildung 5 – Berufsbezeichnung

6.4.3 Alle Nachnamen, keine Doppelten, absteigend sortiert

Um die Nachnamen absteigend sortiert auszugeben, ohne dabei doppelte anzuzeigen, habe ich wieder ein einfaches select ausgeführt, dieses Mal aber mit dem Keyword „DISTINCT“. Das sorgt dafür, dass es keine Duplikate gibt.

```
SELECT DISTINCT lastName
FROM employees
ORDER BY lastName DESC;
```



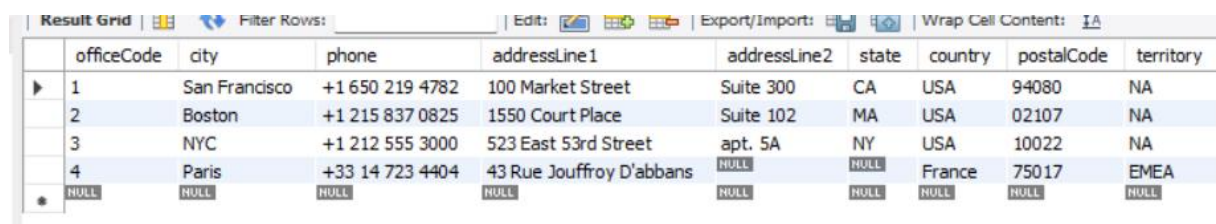
lastName
Vanauf
Tseng
Thompson
Patterson
Nishi
Murphy
Marsh
King
Kato
Jones
Jennings
Hernandez
Gerard
Fixter
Firrelli
Castillo
Bow
Bott
Bondur

Abbildung 6 - Nachnamen anzeigen

6.4.4 USA & Frankreich

Damit ich alle Büros anzeigen kann, die in den USA und in Frankreich sind, habe ich alle offices selected und mit WHERE USA und France gefiltert.

```
SELECT *
FROM offices
WHERE country IN ('USA', 'France');
```



	officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
1	1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
2	2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
3	3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA
4	4	Paris	+33 14 723 4404	43 Rue Jouffroy D'abbans	NULL	NULL	France	75017	EMEA
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Abbildung 7 - USA France

6.4.5 ! Zwischen 20&100 Dollar

Um alle Produkte mit Nummer, Name und Preis anzuzeigen, die nicht zwischen 20-100\$ liegen, habe ich in dem select ein WHERE ... NOT benutzt:

```
SELECT productCode, productName, buyPrice
FROM products
WHERE buyPrice NOT BETWEEN 20 AND 100;
```

	productCode	productName	buyPrice
▶	S10_4962	1962 LanciaA Delta 16V	103.42
	S18_2238	1998 Chrysler Plymouth Prowler	101.51
	S24_2840	1958 Chevy Corvette Limited Edition	15.91
	S24_2972	1982 Lamborghini Diablo	16.24
*	NULL	NULL	NULL

Abbildung 8 - price range

6.4.6 Buchstabe B ausblenden

Alle Angestellten, die nicht mit dem Buchstaben B beginnen, kann man auch WHERE ... NOT LIKE benutzen:

```
SELECT lastName, firstName
FROM employees
WHERE lastName NOT LIKE 'B%';
```

	lastName	firstName
▶	Murphy	Diane
	Patterson	Mary
	Firrelli	Jeff
	Patterson	William
	Jennings	Leslie
	Thompson	Leslie
	Firrelli	Julie
	Patterson	Steve
	Tseng	Foon Yue
	Vanauf	George
	Hernandez	Gerard
	Castillo	Pamela
	Jones	Barry
	Fixter	Andy
	Marsh	Peter
	King	Tom
	Nishi	Mami
	Kato	Yoshimi
	Gerard	Martin

Abbildung 9 - NOT LIKE "B"

6.4.7 Verkaufsvertreter zugewiesen

Damit ich alle Kunden anzeigen kann, die einen Verkaufsvertreter zugewiesen haben, habe ich alle Daten, die ich brauche vom Kunden select und bei salesRepEmployeeNumber auf NOT NULL geprüft:

```
SELECT customerName, country, salesRepEmployeeNumber
FROM customers
WHERE salesRepEmployeeNumber IS NOT NULL;
```

	customerName	country	salesRepEmployeeNumber
▶	Atelier graphique	France	1370
	Signal Gift Stores	USA	1166
	Australian Collectors, Co.	Australia	1611
	La Rochelle Gifts	France	1370
	Baane Mini Imports	Norway	1504
	Mini Gifts Distributors Ltd.	USA	1165
	Blauer See Auto, Co.	Germany	1504
	Mini Wheels Co.	USA	1165
	Land of Toys Inc.	USA	1323
	Euro+ Shopping Channel	Spain	1370
	Value Model Building Co.	Sweden	1504

Abbildung 10 - zugewiesener Verkaufsvertreter

6.4.8 Bestelldaten gruppieren

Damit ich die Bestellnummer, den Bestellstatus und die gesamte Bestellsumme der Tabellen order und orderdetails, gruppiert nach orderNumber anzeigen kann, habe ich einen JOIN benutzt um order und orderdetails, vergleichen zu können, ein SUM für die gesamte Bestellsumme und GROUP BY um nach der orderNumber zu gruppieren.

```
SELECT o.orderNumber, o.status, SUM(od.quantityOrdered * od.priceEach) AS
total
FROM orders o
JOIN orderdetails od ON o.orderNumber = od.orderNumber
GROUP BY o.orderNumber;
```

	orderNumber	status	total
▶	10100	Shipped	10223.83
	10101	Shipped	10549.01
	10102	Shipped	5494.78
	10103	Shipped	50218.95
	10104	Shipped	40206.20
	10105	Shipped	53959.21
	10106	Shipped	52151.81
	10107	Shipped	22292.62

Abbildung 11 - Aufgabe

6.4.9 Angestellte ohne zugewiesene Kunden

Damit ich dieses Mal alle Angestellten bekomme, die keinen Kunden zugewiesen haben, habe ich wieder LEFT (um alle Daten von employee auszugeben und die Daten von customers zu vergleichen) JOIN und WHERE ... NOT NULL benutzt:

```
SELECT e.lastName
FROM employees e
LEFT JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
WHERE c.salesRepEmployeeNumber IS NULL
ORDER BY e.lastName;
```

	lastName
▶	Bondur
	Bow
	Firrelli
	Kato
	King
	Murphy
	Patterson
	Patterson

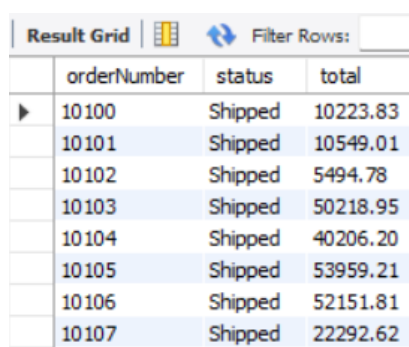
Abbildung 12 - Angestellter ohne Kunde

6.4.10 Bestellungen unter 1500\$ Gesamtsumme

Alle Bestellungen wo die Gesamtsumme nicht über 1500\$ beträgt, habe ich mit diesem Statement ausgegeben:

```
SELECT o.orderNumber, o.status, SUM(od.quantityOrdered * od.priceEach) AS
total
FROM orders o
JOIN orderdetails od ON o.orderNumber = od.orderNumber
WHERE o.status = 'Shipped'
GROUP BY o.orderNumber
HAVING total > 1500;
```

Dabei habe ich mit einem JOIN und select die Tabellen Order und OrderDetails überprüft nach WHERE „orderStatus = versandt“, gruppiert nach Nummer mit GROUP BY und mit HAVING TOTAL auf die mehr als 1500 betragen.



The screenshot shows a 'Result Grid' with a 'Filter Rows' button. The grid contains the following data:

	orderNumber	status	total
▶	10100	Shipped	10223.83
	10101	Shipped	10549.01
	10102	Shipped	5494.78
	10103	Shipped	50218.95
	10104	Shipped	40206.20
	10105	Shipped	53959.21
	10106	Shipped	52151.81
	10107	Shipped	22292.62

Abbildung 13 - Order Gesamtsumme > 1500\$

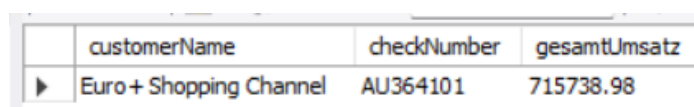
6.4.11 SubSelect – Kunde mit dem meisten Umsatz

Damit ich den Kunden mit dem meisten Umsatz anzeigen konnte, habe ich ein Subselect verwendet:

```
SELECT
    c.customerName,
    p.checkNumber,
    (SELECT SUM(p2.amount)
     FROM payments p2
     WHERE p2.customerNumber = c.customerNumber) AS gesamtUmsatz
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber
ORDER BY gesamtUmsatz DESC
LIMIT 1;
```

SubQuerys sind SQL-Statements, die in einem anderen Statement genestet sind. In dem Beispiel habe ich es so angewendet, dass ich im Select für den Namen und die Nummer, auch einen Select für die Gesamtsumme aller Zahlungen eines Kunden berechnet wird.

Die äußere Abfrage zeigt den Kundennamen und die Zahlungsnummer, während die innere Abfrage, also den SubSelect, alle Beträge aus der Tabelle payments für diesen Kunden zusammenzählt.



The screenshot shows a result grid with the following data:

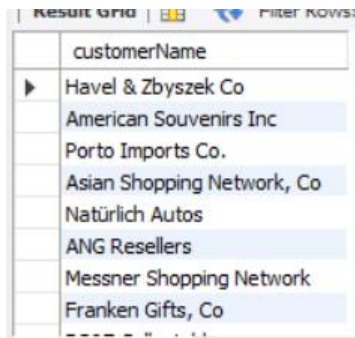
	customerName	checkNumber	gesamtUmsatz
▶	Euro+ Shopping Channel	AU364101	715738.98

Abbildung 14 - SubSelect

6.4.12 Kunden ohne Bestellung

Damit ich die Kunden ohne Bestellungen filtern konnte, habe ich wieder einen SubSelect verwendet. Damit habe ich in der Suche von den CustomerNames alle CustomerNumbers von den Bestellungen gesucht.

```
SELECT customerName
FROM customers
WHERE customerNumber NOT IN (SELECT customerNumber FROM orders);
```



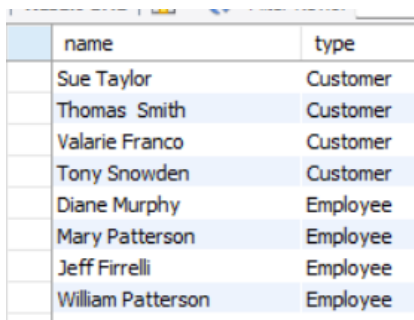
customerName
Havel & Zbyszek Co
American Souvenirs Inc
Porto Imports Co.
Asian Shopping Network, Co
Natürlich Autos
ANG Resellers
Messner Shopping Network
Franken Gifts, Co

Abbildung 15 - SubSelect 2

6.4.13 UNION Operator – Kunden und Angestellte 1 Tabelle

Damit ich Kunden und Angestellte in einer Tabelle anzeigen kann und Vor-&Nachname in einer Spalte stehen habe, habe ich bei beiden Tabellen Vor und Nachname mit Concat (also sozusagen zusammenfügen) Selected und mit UNION in einer Tabelle vereint.

```
SELECT CONCAT(firstName, ' ', lastName) AS name, 'Customer' AS type
FROM customers
UNION
SELECT CONCAT(firstName, ' ', lastName) AS name, 'Employee' AS type
FROM employees;
```



name	type
Sue Taylor	Customer
Thomas Smith	Customer
Valarie Franco	Customer
Tony Snowden	Customer
Diane Murphy	Employee
Mary Patterson	Employee
Jeff Firrelli	Employee
William Patterson	Employee

Abbildung 16 - Customer - Employee

6.5 Neue Tabelle suppliers

Den Table Suppliers habe ich mit dem CREATE Statement aus dem Script erstellt:

```
CREATE TABLE suppliers (
  supplierNumber INT AUTO_INCREMENT,
  supplierName VARCHAR(50) NOT NULL,
  phone VARCHAR(50),
  addressLine1 VARCHAR(50),
  addressLine2 VARCHAR(50),
  city VARCHAR(50),
  state VARCHAR(50),
  postalCode VARCHAR(50),
```

```
country VARCHAR(50),
customerNumber INT,
PRIMARY KEY (supplierNumber)
);
```

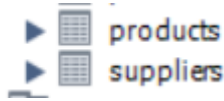


Abbildung 17 - suppliers table

6.5.1 Tabelle mit customers Daten befüllen

Die Daten aus der Customers Tabelle habe ich so in die suppliers Tabelle geschrieben:

```
INSERT INTO suppliers (supplierName, phone, addressLine1, city, country)
SELECT customerName, phone, addressLine1, city, country
FROM customers;
```

Mit dem INSERT INTO kann ich festlegen in welche Tabelle/Felder ich die folgenden Daten vom SELECT einfügen möchte.

	supplierNumber	supplierName	phone	addressLine1	addressLine2	city	state	postalCode	country	customerNum
1		Atelier graphique	40.32.2555	54, rue Royale		Nantes			France	
2		Signal Gift Stores	7025551838	8489 Strong St.		Las Vegas			USA	
3		Australian Collectors, Co.	03 9520 4555	636 St Kilda Road		Melbourne			Australia	
4		La Rochelle Gifts	40.67.8555	67, rue des Cinquante Otages		Nantes			France	
5		Baane Mini Imports	07-98 9555	Erling Skakkes gate 78		Stavern			Norway	
6		Mini Gifts Distributors Ltd.	4155551450	5677 Strong St.		San Rafael			USA	
7		Havel & Zbyszek Co	(26) 642-7555	ul. Filtrowa 68		Warszawa			Poland	

Abbildung 18 - suppliers Daten - INSERT INTO

6.6 Maildomains aktualisieren – replace

Damit ich die Maildomains die zurzeit „@mynextmail.org“ gesetzt haben auf „@lbs4.salzburg.at“ zu setzen, habe ich mit UPDATE employees, die Angestellten-Tabelle bearbeitet und mit SET E-Mail REPLACE die Daten abgeändert. Das WHERE setzt, dass die Daten nur bei den angegebenen Bedingungen geupdated werden sollten.

```
UPDATE employees
SET email = REPLACE(email, '@mynextmail.org', '@lbs4.salzburg.at')
WHERE jobTitle = 'Sales Rep' AND officeCode = 6;
```

Wichtig: Dafür musste man die Safe-Updates disable, mit einem einfachen Befehl:

```
SET SQL_SAFE_UPDATES = 0;
```

	email
1	afixter@lbs4.salzburg.at
2	pmarsh@lbs4.salzburg.at
3	tking@lbs4.salzburg.at

Abbildung 19 - updated E-Mail

6.7 Kunden ohne SalesRep zufällige zuweisen

Da es 22 Kunden ohne Betreuer gibt, habe ich mit einem Update den Kunden eine zufällige Betreuernummer zugewiesen. Dabei habe ich einen SubQuery verwendet und die rand()-Funktion:

```
UPDATE customers
SET salesRepEmployeeNumber = (
    SELECT employeeNumber
```

```

FROM employees
ORDER BY RAND()
LIMIT 1
)
WHERE salesRepEmployeeNumber IS NULL;

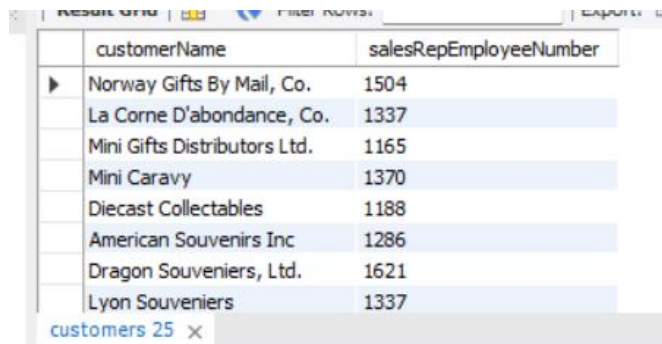
```

Das Ergebnis habe ich mit ORDER BY RAND() überprüft, damit sind die Ergebnisse random ausgegeben.

```

SELECT customerName, salesRepEmployeeNumber
FROM customers
ORDER BY RAND();

```



customerName	salesRepEmployeeNumber
Norway Gifts By Mail, Co.	1504
La Corne D'abondance, Co.	1337
Mini Gifts Distributors Ltd.	1165
Mini Caravy	1370
Diecast Collectables	1188
American Souvenirs Inc	1286
Dragon Souvenirs, Ltd.	1621
Lyon Souvenirs	1337

customers 25 x

Abbildung 20 - rand() salesRep Nummer

7 Erweiterung

Als Erweiterung für diese Übung habe ich ein SQL-Statement mit einer SubQuery und der Funktion AVG() erstellt.

Damit wollte ich herausfinden, welcher Verkaufsmitarbeiter den höchsten durchschnittlichen Umsatz erzielt. Mit dieser Erweiterung wollte ich auch die vorherige Aufgabe zu den Views einbauen.

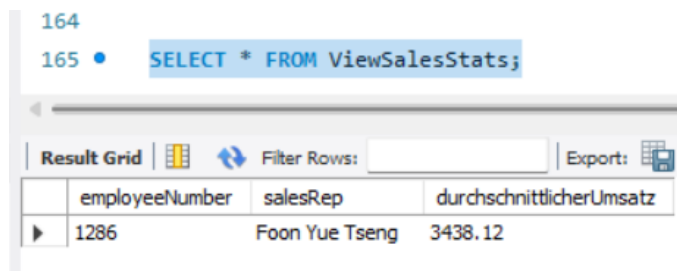
Das Statement erstellt eine View, die für jeden Mitarbeiter den durchschnittlichen Bestellwert berechnet:

```

CREATE VIEW ViewSalesStats AS
SELECT
    e.employeeNumber,
    CONCAT(e.firstName, ' ', e.lastName) AS salesRep,
    ROUND(AVG(od.quantityOrdered * od.priceEach), 2) AS
durchschnittlicherUmsatz
FROM employees e
JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
JOIN orders o ON c.customerNumber = o.customerNumber
JOIN orderdetails od ON o.orderNumber = od.orderNumber
GROUP BY e.employeeNumber
HAVING AVG(od.quantityOrdered * od.priceEach) = (
    SELECT MAX(avgSales)
    FROM (
        SELECT AVG(od2.quantityOrdered * od2.priceEach) AS avgSales
        FROM employees e2
        JOIN customers c2 ON e2.employeeNumber = c2.salesRepEmployeeNumber
        JOIN orders o2 ON c2.customerNumber = o2.customerNumber
        JOIN orderdetails od2 ON o2.orderNumber = od2.orderNumber
        GROUP BY e2.employeeNumber
    ) AS sub
);

```

Der SubQuery wird anschließend verwendet, um den höchsten Durchschnittswert (MAX) aus dieser View zu ermitteln. Mit der Funktion ROUND() habe ich das Ergebnis auf zwei Nachkommastellen begrenzt, um eine übersichtliche Ausgabe zu erhalten.



The screenshot shows a SQL query editor with a query window at the top and a result grid below it. The query window contains the text 'SELECT * FROM ViewSalesStats;'. The result grid has three columns: 'employeeNumber', 'salesRep', and 'durchschnittlicherUmsatz'. The first row of data shows '1286' for employeeNumber, 'Foon Yue Tseng' for salesRep, and '3438.12' for durchschnittlicherUmsatz.

```
164
165 • SELECT * FROM ViewSalesStats;
```

	employeeNumber	salesRep	durchschnittlicherUmsatz
▶	1286	Foon Yue Tseng	3438.12

Abbildung 21 – Erweiterung

(Bei der Erweiterung hatte ich durch die Komplexität kleine Schwierigkeiten und hab mir bei den Feinschliffen und Ausbesserungen mit ChatGPT weitergeholfen.)

8 Einsatzgebiet

Die Datenbankanalyse wird sehr häufig in Unternehmen, Onlineshops oder Verwaltungssystemen genutzt. Das Erstellen und Bearbeiten von Datenbanken, das Definieren von Fremdschlüsseln und das Durchführen von Abfragen mit JOINS, SubSelects oder UNIONS sind grundlegende Aufgaben, die einem in der DB-Verwaltung helfen.

9 Erkenntnisse

Bei dieser Laborübung habe ich einige alte Kenntnisse wieder aufgefrischt. Sowie:

- Verschiedene SELECTs
- JOINS
- GROUPING % WHERE
- rand()

Zusätzlich habe ich neue Funktionen kennengelernt & benutzt:

- SubQuerys
- UNIONS
- REPLACE

Unterschrift:



Mit meiner Unterschrift bestätige ich, dass ich alle Texte und Bilder eigenständig erstellt habe, wenn nicht Quellen angegeben sind.

10 Abbildungsverzeichnis

Abbildung 1 - meow	3
Abbildung 2 - reverse engineering mysql workbench.....	4
Abbildung 3 - ER Diagramm.....	4
Abbildung 4 - 5 Personen Abfrage.....	5
Abbildung 5 – Berufsbezeichnung.....	5
Abbildung 6 - Nachnamen anzeigen.....	6
Abbildung 7 - USA France	6
Abbildung 8 - price range	7
Abbildung 9 - NOT LIKE "B"	7
Abbildung 10 - zugewiesener Verkaufsvertreter	8
Abbildung 11 - Aufgabe.....	8
Abbildung 12 - Angestellter ohne Kunde	8
Abbildung 13 - Order Gesamtsumme > 1500\$.....	9
Abbildung 14 - SubSelect.....	9
Abbildung 15 - SubSelect 2.....	10
Abbildung 16 - Customer - Employee.....	10
Abbildung 17 - suppliers table.....	11
Abbildung 18 - suppliers Daten - INSERT INTO.....	11
Abbildung 19 - updated E-Mail.....	11
Abbildung 20 - rand() salesRep Nummer	12
Abbildung 21 – Erweiterung.....	13